

Đây là kiến trúc mới đề xuất:

Query + History -> Query Decomposer (Tách thành N sub-queries) -> Parallel RAG Retrieval (Dense + Sparse + SQL -> RRF) -> Synthesis LLM (Suy luận trên context -> câu trả lời) ->

Response tổng hợp đa intent.

Lớp dữ liệu: 4 lớp tách biệt rõ: Doc store, Vector Knowledge, History, Trace độc lập

RAG-first; Multi-intent; Memory

Sơ đồ chi tiết pipeline mới với Query Decomposition và Parallel RAG:

1. User query + conversation history -> 2. Query Decomposer (LLM) (phân tích -> N sub-queries có intent riêng) -> 3.1. Sub-query 1: FAQ “ê buốt -> dịch vụ nào?” -> 4.1. RAG retrieval (dense + sparse + RRF); 3.2. Sub-query 2: Service “Giá trị liệu ê buốt” -> 4.2. RAG+SQL (services + chunks); 3.3. Clinic “Giờ mở cửa phòng khám?” -> 4.3. Direct SQL (clinic_info table)
-> 5. Context Merger + Reranker (Gộp kết quả, dedup, rank, giới hạn tokens)
-> 6. Synthesis LLM – đây là phần suy luận (Dùng toàn bộ context để reasoning + trả lời; Không format cứng – tự quyết định cấu trúc câu trả lời) -> 6.1 History (lưu turn) -> 1.
-> 7. Response tổng hợp

Vấn đề 1: Không có RAG thực sự

Triết lý hiện tại là **SQL-first, RAG-as-fallback**. Cần đảo ngược thành **RAG-first, SQL-as-enrichment**. Nghĩa là với mọi query, bước đầu tiên luôn là semantic retrieval từ vector store, rồi mới merge thêm dữ liệu có cấu trúc từ SQL vào context. LLM sau đó nhận toàn bộ context đó để suy luận và trả lời, thay vì chỉ format lại kết quả SQL.

Thay đổi cụ thể trong app/retrieval/:

python

```
# Cũ: chọn một trong hai
```

```
if intent == "PRODUCT_DETAIL":
```

```
    return direct_sql_lookup(entity)
```

```
# Mới: luôn lấy cả hai, merge vào context
```

```
semantic_hits = hybrid_retriever.search(query, top_k=10)
```

```
structured_hits = sql_enricher.enrich(semantic_hits) # bổ sung price, asset, metadata
```

```
context = context_builder.merge(semantic_hits, structured_hits)
```

```
return synthesis_llm.generate(context, query)
```

Vấn đề 2: Single intent — pipeline cứng

Cần thêm bước **Query Decomposition** trước router hiện tại. LLM nhỏ (cùng model router 3b) nhận query và trả về danh sách sub-queries:

```
python
# app/retrieval/query_decomposer.py ← file mới
DECOMPOSE_PROMPT = """
Phân tích câu hỏi sau thành các câu hỏi con độc lập.
Mỗi câu hỏi con có một intent duy nhất.
Tối đa 3 câu hỏi con. Nếu câu hỏi đơn giản, chỉ trả về 1.
```

Output JSON:

```
{"sub_queries": [
  {"query": "...", "intent": "FAQ", "priority": 1},
  {"query": "...", "intent": "SERVICE_DETAIL", "priority": 2}
]}
```

Sau khi có sub-queries, pipeline chạy retrieval song song (asyncio.gather) cho từng sub-query, rồi merge context trước khi đưa vào LLM synthesis.

Vấn đề 3: Quá nhiều bảng, khó mở rộng

Không cần xóa bảng hiện tại, nhưng cần **tách logic** thành 4 lớp rõ ràng:

Lớp	Bảng thuộc về	Vai trò
Document store	documents, chunks, tables, table_rows	Lưu dữ liệu gốc và embedding
Knowledge base	products, services, faqs, clinic_info	Business data có cấu trúc
Asset store	assets, chunk_assets	Media files
Observability	rag_traces, rag_trace_steps, ingestion_runs	Chỉ dùng nội bộ

Thêm **conversation memory** tách biệt hoàn toàn — hiện tại không có bảng nào lưu history session:

```
sql
-- Bảng mới cần thêm
CREATE TABLE conversation_sessions (
  session_id UUID PRIMARY KEY,
  created_at TIMESTAMP,
  metadata JSONB
);

CREATE TABLE conversation_turns (
  turn_id UUID PRIMARY KEY,
  session_id UUID REFERENCES conversation_sessions,
  role TEXT,      -- 'user' | 'assistant'
  content TEXT,
  intent TEXT,
```

```
entities JSONB,  
resolved_ids JSONB,  
created_at TIMESTAMP  
);
```

Vấn đề 4: Không có tính suy luận

Đây là vấn đề ở **prompt của Generation LLM**. Hiện tại prompt yêu cầu model trả về JSON có cấu trúc cứng, không có không gian để suy luận. Cần tách làm hai bước:

Bước 1 — Reasoning (ẩn): LLM suy luận trên context, viết ra "think step"

Bước 2 — Output (hiện): LLM format thành JSON theo schema

Với Ollama (local model), cách đơn giản nhất là dùng <think> tag trong prompt:

```
python  
SYNTHESIS_PROMPT = """  
Bạn là trợ lý tư vấn nha khoa.
```

```
<retrieved_context>  
{context}  
</retrieved_context>
```

```
<conversation_history>  
{history}  
</conversation_history>
```

Câu hỏi: {query}

Trước khi trả lời, hãy suy nghĩ (trong thẻ <think>):

- Người dùng thực sự cần gì?
- Context nào liên quan nhất?
- Có thông tin thiếu không? Cần hỏi lại gì?
- Nếu là câu hỏi y tế, cần disclaimer gì?

```
<think>  
[LLM tự viết reasoning ở đây — không hiện cho user]  
</think>
```

Sau đó trả về JSON:

```
{schema}  
"""
```

Parser sẽ strip phần <think>...</think> trước khi validate JSON output.

Điểm quan trọng nhất cần làm ngay là **Query Decomposition + RAG-first** — hai thay đổi này sẽ thay đổi cảm nhận của người dùng rõ ràng nhất mà không cần rewrite toàn bộ hệ thống.